

Programación en C: Fundamentos

Tema 1

ÍNDICE

01 Introducción

02 Tipos de datos

03 Entrada y salida de datos

04 Expresiones y operaciones

MOTIVACIÓN

- ¿Qué necesita un ordenador para ser útil?
- ¿Cuál es el lenguaje de un ordenador?
- ¿De qué están hechas las aplicaciones informáticas?
- ¿Cómo se ejecuta un programa en un ordenador?



01

Introducción

¿Qué es un Ordenador?

- Máquina electrónica diseñada para manipular y procesar datos, capaz de desarrollar complejas operaciones a gran velocidad siguiendo un programa de instrucciones.
- Todo aparato destinado a procesar información, entendiendo por proceso, las sucesivas fases, manipulaciones o transformaciones que sufre la información para resolver un problema determinado, siguiendo las instrucciones de un programa.
- Entre sus características podemos citar:
  - Son programables.
  - Versatilidad. Permiten realizar diversos trabajos
  - Interactividad. Mantienen diálogos con los usuarios y actúan en función de sus órdenes
  - Alta Capacidad de almacenamiento.
  - Rapidez.

Elementos de un ordenador

Ordenador

Distinguimos 4 partes

Software

Los programas

Hardware

Los dispositivos físicos

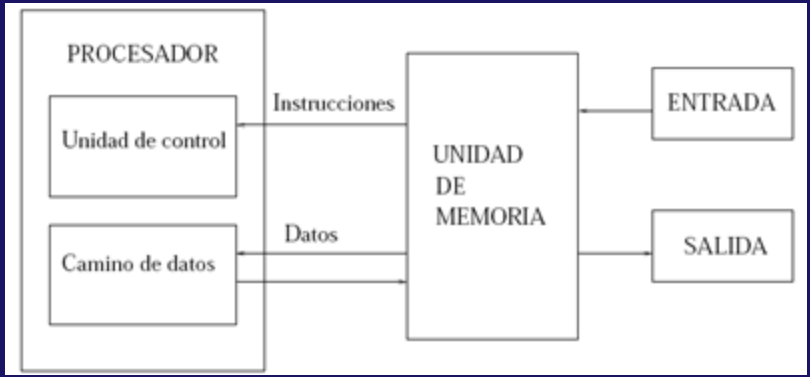
Datos

La información que se procesa

Personas

Los usuarios que se benefician del sistema

Unidades funcionales básicas



- **Unidad central de procesamiento**: controla el funcionamiento del computador y lleva a cabo sus funciones de procesamiento de datos.
- **Memoria**: almacena datos e instrucciones.
- **E/S**: transfiere datos entre el computador y su entorno.
- **Sistema de interconexión**: es un mecanismo que proporciona la comunicación entre la CPU, la memoria principal y la E/S.

Hardware: Memoria principal

- Gran velocidad para escribir y leer de ella
- Un programa ejecutable debe estar almacenado en ella
- Está dividida en "palabras de memoria". Cada palabra tiene asociada una dirección
- Tipos:
  - ROM – sólo lectura y permanente
  - RAM – lectura/escritura y volátil

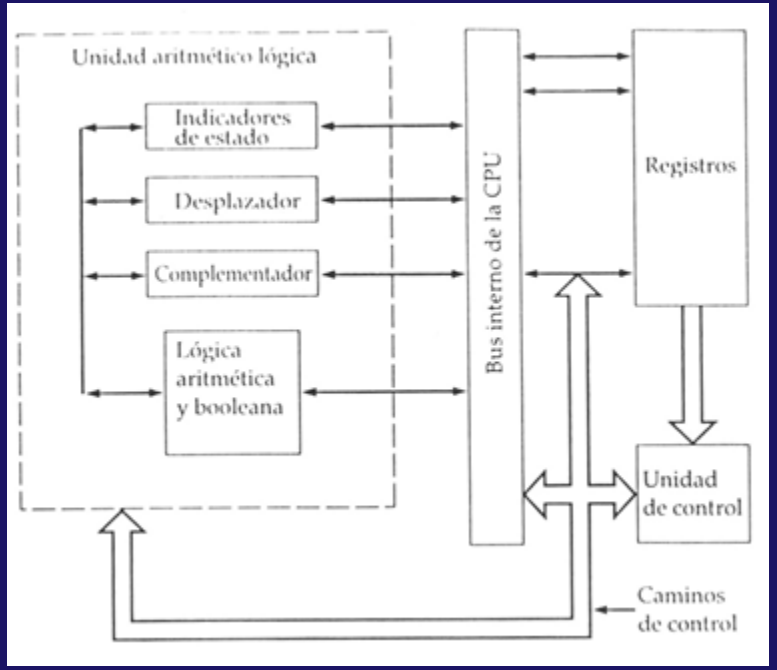
Dirección	Contenido
0	3745
1	2356
2	3725
3	4832
4	2437
5	4326
.	.
.	.
255	3456

Hardware: Medidas de memoria

- Un bit es la unidad de información más pequeña. Puede tener sólo dos valores: 1 ó 0
- La capacidad de una memoria se expresa con múltiplos de Byte
- 1 Byte = 8 bits

Unidad	Valor Aprox. (en bytes)
Kilobyte (KB)	1000
Megabyte (MB)	1000000
Gigabyte (GB)	1000000000
Terabyte (TB)	1000000000000

Hardware: la CPU



- Los programas se gestionan en un microprocesador
- **Micro-procesador**: procesador fabricado en un único circuito integrado
- Unidad Central de Proceso (Procesador):
  - **Unidad de control** (UC): gestiona el funcionamiento de la CPU → computador.
  - **Unidad aritmético-lógica** (ALU): lleva a cabo las funciones de procesamiento de datos (+, -, /, *, <, >, >=, etc.).
  - **Registros**: almacenamiento interno.
  - **Interconexiones CPU**: Son mecanismos que proporcionan comunicación entre la unidad de control, la ALU y los registros.
- Precisa de un **reloj** para coordinar todas las operaciones

El “Software”



¡¡ Y mis propios programas !!

Software del ordenador

- El software del ordenador se puede clasificar esencialmente en dos tipos:



El Sistema Operativo

- Gestiona los recursos del ordenador
- Facilita el uso del ordenador:
  - Hace transparente al usuario las características de los dispositivos
  - Proporciona una interface de usuario
- Servicios importantes de un SO:
  - Sistema de ficheros
  - Entrada/salida con distintos dispositivos
  - Ejecución de programas
  - Medidas de seguridad
- Durante el arranque se carga en memoria, y permanece residente todo el tiempo

Programa: Conceptos básicos

- La solución a un problema debe establecerse en términos de una secuencia de pasos computacionales (programa).
- Especificación pasos computacionales → notaciones sistemáticas conocidas como lenguajes de programación.
- La programación es la tarea de realizar programas para resolver problemas mediante un ordenador.
- Existen metodologías que permiten que la programación sea lo más eficaz en cuanto al desarrollo y mantenimiento.



Los ordenadores son tontos

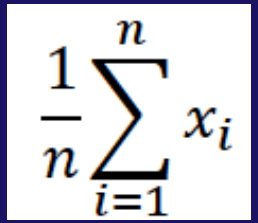
- Ejecutan rápida y precisamente operaciones lógicas y matemáticas.
- Cumplen obedientemente lo que les decimos (¡no siempre igual a lo que queremos!).
  - Errores pequeños pueden provocar fallos graves.
- Un programa de ordenador es un software que implementa un algoritmo.

¿Qué es un algoritmo?

Definición: *método para resolver un problema mediante una serie de pasos:*

1. ***Precisos***: indicar el orden de ejecución en cada paso.
2. ***Definidos:*** si el algoritmo se prueba dos veces, en estas dos pruebas, se debe obtener el mismo resultado.
1. ***Finitos:*** el algoritmo tiene un número determinado de pasos.

Los algoritmos se pueden expresar por fórmulas, diagramas de flujo, y pseudo códigos. Está última representación es la mas utilizada por su sencillez y parecido al lenguaje humano.



Sumar todos los valores y dividirlos por su cantidad

Algoritmo MAL

- ENTRADA: macarrones, sal, aceite, tomate frito, agua
  1. Hervir el agua
  2. Mezclar agua hirviendo con sal y aceite
  3. Añadir macarrones
  4. Mientras (macarrones no estén blandos)
     - Esperar
  5. Escurrir agua
  6. Mezclar macarrones con tomate frito
- SALIDA: macarrones con tomate

Programa para sumar dos números

Código del programa en lenguaje natural:

1. Mostrar un mensaje para que el usuario proporcione el primer número
2. Leer el número y guardarlo en memoria
3. Mostrar un mensaje para que el usuario proporcione el segundo número
4. Leer el número y guardarlo en memoria
5. Calcular la suma de ambos números
6. Mostrar un mensaje al usuario con el resultado calculado

Lenguajes de programación

- Conjunto de símbolos y reglas sintácticas y semánticas que definen su estructura y el significado de sus elementos y expresiones.
    - Controla el comportamiento físico y lógico de una máquina.
- Permite al programador especificar de *manera precisa* sobre qué datos debe operar una computadora, cómo deben ser almacenados o transmitidos y qué acciones debe tomar bajo una variada gama de circunstancias.
- Los principales tipos de lenguajes utilizados son:
    - Lenguaje Máquina.
    - Lenguaje de bajo Nivel o Ensamblador.
    - Lenguajes de Alto Nivel.

Lenguaje máquina

- Lenguaje que puede ser interpretado directamente por los circuitos internos de la máquina.
- Está basado en instrucciones compuestas por sentencias de ceros (0) y unos (1).
- **Ventajas**:
  - No requiere traducción, la máquina lo interpreta.
  - Opera directamente con la máquina.
  - Puede ser eficiente en el manejo de los recursos.
- **Desventajas**:
  - Difícil de aprender, implementar y mantener.
  - Es único y particular para cada procesador



Lenguaje de bajo nivel (ASM)

- Lenguaje de Bajo Nivel o Ensamblador es un lenguaje fuente para codificar sentencias simbólicas

- Hace uso de nemotécnicos para representar instrucciones de máquina.

- Un ensamblador convierte cada instrucción en instrucciones de lenguaje de máquina.

Lenguaje de bajo nivel (ASM)

- Los lenguajes ensamblador se usan principalmente en aplicaciones de tiempo real, control de procesos y de dispositivos electrónicos
- Ventajas:
  - No es tan complicado como el lenguaje de máquina.
  - Eficiente en el manejo de recursos.
- Desventajas:
  - Cada procesador tiene su propio juego de instrucciones en ensamblador
  - Dependencia total de la máquina lo que impide la transportabilidad de los programas.
  - Debe ser traducido – ensamblado.

Lenguaje de alto nivel

- Es un lenguaje orientado al problema o procedimiento.
- Son los más cercanos al lenguaje humano.
- Ejemplos: C, Pascal, FORTRAN. Lenguajes de Programación Orientado a Objetos (OOPL): C++, Python, Java, etc.
- **Ventajas**:
  - Repertorio de instrucciones amplio, potente, y fácilmente asimilable
  - **Son independientes del procesador**
  - Son más fáciles de aprender, implementar y mantener.
  - Tiene un carácter genérico
- **Desventajas**:
  - Menos eficiente en el manejo de recursos
  - **Requiere un proceso de traducción**

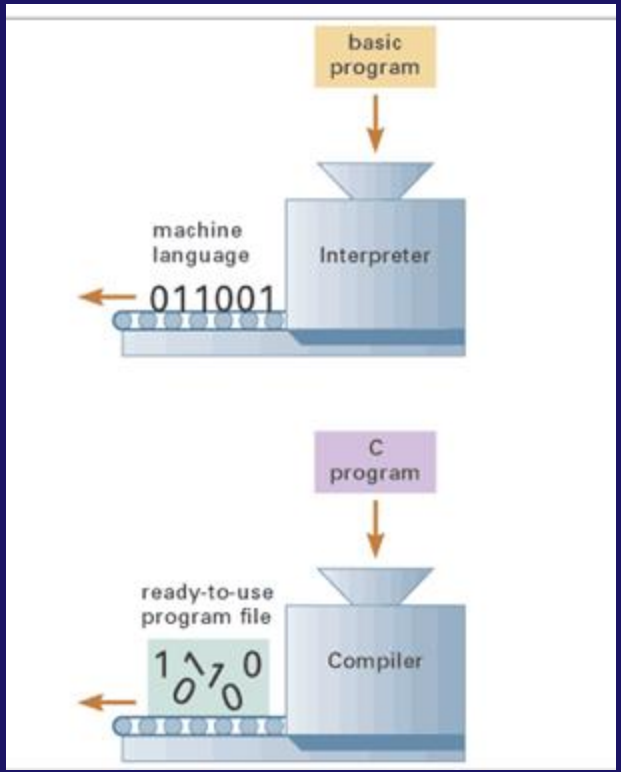


Compilación e interpretación

- Tanto los lenguajes de alto nivel como los de bajo nivel, no son entendibles directamente por la máquina.
- Necesitan ser traducidos a lenguaje de máquina
- Es necesario disponer de un "traductor" con el lenguaje de máquina para que el programa sea ejecutable.
- Existen dos tipos fundamentales de traductores:
  - Compiladores
  - Interpretes

Traductores

- Intérprete:
  - Analizan, traducen y ejecutan una a una las instrucciones del programa fuente;
  - No se analiza una instrucción hasta que la anterior se haya ejecutado.
  - Los intérpretes no generan programa objeto.
- Compilador:
  - Software que traduce el programa inicial (programa fuente) escrito en un lenguaje de alto nivel, a un programa completo (programa objeto) en lenguaje de máquina.



Entornos Integrados de Desarrollo

- Un "IDE" es un software que incorpora todas las **herramientas** necesarias para el desarrollo de nuestros programas:
  - Editor
  - Compilador
  - "Linker"
  - Depuración
  - Ejecución

- En prácticas: Code::Blocks



- En casa: Code::Blocks ; MS Visual Studio ; X-Code; NetBeans...

Primer programa en C

```c
// mi primer programa en C

#include <stdio.h>


int main()
{
    printf("Qué bonito es C!\n");

    return 0;
}
```

Comentarios

Se escriben entre **/*** y ***/** o luego de **//**.

Mejoran la legibilidad del programa y son

directiva del preprocesador

#include <stdio.h> indica al preprocesador que incluya el contenido del fichero **<stdio.h>**, donde se definen las

Los programas en C contienen una o mas

return es la forma de salir de una función.

return 0, en este caso, indica que el programa finalizó correctamente.

Una llave **{** comienza el cuerpo de cada función y una llave **}** lo finaliza.

Imprime el *string* de caracteres entre comillas.
Toda esta línea, incluyendo el *punto y coma* (**;**), se denomina *sentencia.*
Las sentencias terminan con *punto y coma* (**;**)

Comentarios del programa

/* Los comentarios sirven para aclarar que hace el
codigo de un programa.
Un comentario NO FORMA PARTE DEL PROGRAMA.
Un comentario solo sirve para poner notas y aclarar lo que hace el programa, para que otro programador pueda leerlo mejor
*/

/*
Esto es un comentario de varias líneas. Comienza
con un /* y siempre termina
cuando se encuentre el primer "asterisco-barra". Así: */

// Esto es un comentario de una única línea, con //

Cómo organizar un programa

1. Se obtienen los datos de entrada
2. Se procesan para obtener el resultado (datos de salida)
3. Se devuelven los datos de salida

- Un dato es una representación de la propiedad de un objeto, que puede ser utilizado en la realización de cálculos o toma de decisiones.

    - Datos **numéricos** (1, 3.14), **caracteres** ('a', '8'), **cadenas de caracteres** (“hola”, “comienzo”), **booleanos** (true, false)

Ejemplo de programa (sumanumeros.c)

```c
/* Programa que toma dos números por teclado y hace su suma*/
#include <stdio.h>

int main() {
  //declaración de variables
  int a, b, suma = 0;
  //obtención de datos de entrada
  printf("Ingrese dos números: ");
  scanf("%d %d", &a, &b);

  //operaciones con los datos
  suma = a + b;

  //salida de datos
  printf("La suma de los dos números es: %d\n", suma);

  return 0;
}
```

02

Tipos de datos

Tipos de datos

- Datos → objetos de información con los que un programa trabaja.
- Todos los datos tienen un tipo asociado
  - Indica con qué información trabajaremos.
- Ejemplos:
  - El DNI de una persona es un número entero
  - El Ph de una solución es un número con decimales
  - El título de un libro es una lista de letras

- Tipos de datos usuales: numéricos, caracteres, lógicos.

Tipos de datos

- Objetivos de los tipos de datos:
  - Detectar errores de operaciones aritméticas en los programas
  - Determinar como ejecutar las operaciones

- Todos los datos de un programa deben tener un nombre o identificador.

Tipos de datos (tamaño típico)

Tipo de dato	Descripción	Nº de Bytes	Rango (signed)	Rango (unsigned)
(unsigned) short	Entero corto	2	-32768 a +32767	0 a 65535
(unsigned) int	Entero	2	−32767 a +32767	0 a 65535
(unsigned) long	Entero largo	4	-2147483648 a +2147483647	0 a 4294967295
(unsigned) char	Carácter	1	-128 a 127	0 a 255
float	Real (coma flotante)	4	+: 3.4E-38 a 3.4E38 -: -3.4E-38 a -3.4E38	-
double	Real (coma flotante doble precisión)	8	+: 1.7E-308 a 1.7E308 -: -1.7E-308 a -1.7E308	-
long double	Real (doble largo)	16	+: 3.4E-4932 a 1.1E4932 -: -3.4E-4932 a -1.1E4932	-
bool	Dato lógico	1	0, 1 (true / false)	-

Más sobre los enteros

- El rango de un entero (integer) es un subconjunto del conjunto matemático Z.
- La cardinalidad dependerá del número de bits (r) que cada compilador utiliza para su almacenamiento.
- Los compiladores suelen ofrecer distintos tipos enteros. En C: short, int, long, etc. El más usado es int.
- El estándar de C no obliga a los compiladores a usar un tamaño determinado.
- Lo usual es que un int ocupe 16 bits. El rango sería [-32767, +32767]
- Existe el tipo long int que ocupa 32 bits. Su rango, por lo tanto, será:

$$\left[-\frac{2^{32}}{2}, \frac{2^{32}}{2} - 1\right] = [-2\,147\,483\,648, 2\,147\,483\,647]$$

Más sobre los enteros

- Cuando necesitemos un entero mayor, podemos usar el tipo `long long int`. También puede usarse la forma abreviada del nombre: `long long`.
- Es un entero de 64 bits y es estándar desde C99. El rango sería:

$$[-9\,223\,372\,036\,854\,775\,808, 9\,223\,372\,036\,854\,775\,807]$$

Desbordamiento

- ¿Qué ocurre en el siguiente código?

```c
int main(){
    int entero;
    entero = 2147483647 + 1;
    printf("%d", entero);
    return 0;
}
```

- Los literales 2147483647 y 1 son ambos int. Por lo tanto, el compilador entiende que la expresión 2147483647 + 1 es también int, pero el resultado se sale del rango.
- Se produce un error lógico denominado desbordamiento aritmético (arithmetic overflow) y el resultado de evaluar la expresión es un valor indeterminado.

Más sobre reales

- Un dato de tipo real (float) tiene como rango un subconjunto finito de R
- El separador decimal es el **punto** → **3.141592**

- C ofrece distintos tipos para representar valores reales. Principalmente, float (usualmente 32 bits) y double (usualmente 64 bits).
  - El literal 3 es un entero
  - El literal 3.0 es un real

Más sobre reales

- Por ejemplo, 0.1011 representa:

$$1 * \frac{1}{2} + 0 * \frac{1}{4} + 1 * \frac{1}{8} + 1 * \frac{1}{16} = 0.6875$$

- Los tipos float y double tienen precisión limitada
  - No pueden representar infintos decimales (no hay memoria infinita)
- La precisión limitada induce aproximaciones en la representación de los números reales...
- ... y acumulación de errores al operar repetidamente con ellos

Más sobre reales

```
double valor_real;

// Datos de tipo double con menos de 15 cifras
// (en su representación decimal)

valor_real = 11.0;                                  
// Almacena: 11.0                                   Correcto
valor_real = 1.1;                                   
// Almacena: 1.1000000000000001                     Problema de redondeo

// Datos de tipo double con más de 15 cifras
// (en su representación decimal)

valor_real = 10000000000000000000100.0;             
// Almacena: 10000000000000000000000.0              Problema de precisión
valor_real = 0.10000000000000000000009;             
// Almacena: 0.10000000000000001                    Problema de precisión
valor_real = 1.00000000000000000000009;             
// Almacena: 1.0                                    Problema de precisión
valor_real = 10000000001.000000000004;              
// Almacena: 10000000001.0                          Problema de precisión
```

El tipo lógico (bool)

- En C89 no existía el tipo lógico
  - Era necesario utilizar int para simular su comportamiento: el valor 1 indica el valor true y el valor 0 el valor false

```c
#include <ctype.h>

int main() {
    int es_alfa, es_alfanum, es_digito_numerico, es_positivo;

    es_alfa = isalpha('3'); /* Asigna 0 a es_alfa */
    es_alfanum = isalnum('3'); /* Asigna 1 a es_alfanum */
    es_digito_numerico = isdigit('3'); /* Asigna 1 a es_digito_numerico */
    es_positivo = 4 > 0; /* Asigna 1 a es_positivo */

}
```

El tipo lógico (bool)

- En la asignatura simularemos un tipo bool usando la directiva #define

```c
#include <ctype.h>

#define bool int
#define true 1
#define false 0

int main() {
    bool es_alfa, es_alfanum, es_digito_numerico, es_positivo;

    es_alfa = isalpha('3'); /* Asigna 0 a es_alfa */
    es_alfanum = isalnum('3'); /* Asigna 1 a es_alfanum */
    es_digito_numerico = isdigit('3'); /* Asigna 1 a es_digito_numerico */
    es_positivo = 4 > 0; /* Asigna 1 a es_positivo */

}
```

El tipo lógico (bool)

- Desde la introducción del estándar C99 la biblioteca stdbool.h permite utilizar el tipo bool directamente:

```c
#include <ctype.h>
#include <stdbool.h>

int main() {
    bool es_alfa, es_alfanum, es_digito_numerico, es_positivo;

    es_alfa = isalpha('3'); /* Asigna 0 a es_alfa */
    es_alfanum = isalnum('3'); /* Asigna 1 a es_alfanum */
    es_digito_numerico = isdigit('3'); /* Asigna 1 a es_digito_numerico */
    es_positivo = 4 > 0; /* Asigna 1 a es_positivo */

}
```

Literales

- Un literal es una especificación de un valor concreto de un tipo de dato. Se utilizan en el lado derecho de una asignación.
- Ejemplos

Valor	Tipo Literal
2	entero
45.2	real
'c'	caracter: se escribe entre ' '
"Hola"	cadena de caracteres: se escribe entre " "
true	booleano
3.49e4	equivale a 3.49×10^{4} (34900.0)
5.89e-6	equivale a 5.89×10^{-6} (0.00000589)

Variables

- Es equivalente a una "variable" en el contexto matemático.
- Al ser un dato de un programa, debe tener un nombre (identificador) y un tipo.
- "Físicamente" es una porción de memoria donde se puede almacenar un dato compatible con su tipo.
- Las variables tienen que declararse antes de su uso siguiendo el esquema:

```
tipo nombre_de_variable;
```

- Ejemplos →

```
int velocidad; //No olvidar el ";"
double ph;
char inicial;
```

Inicializar variables

- Antes de usar una variable, es necesario darle un valor → inicialización

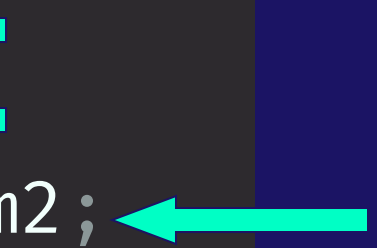
- Forma usual: mediante el **operador** de asignación.

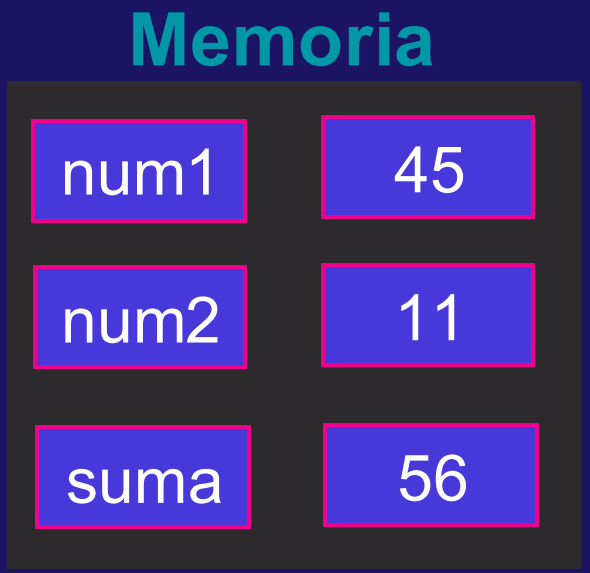
```
int nro;
nro = 45; //asignamos el valor 45 a la
variable nro
```

- En una operación de asignación, primero se evalúa la expresión del lado derecho y luego se almacena el valor resultante en la variable indicada en el lado izquierdo.

Variables: asignar valores

```
int num1, num2, suma;
num1 = 45;
num2 = 11;
suma = num1 + num2;
```






```
suma = 45 + 11;
suma = 56;
```



Regla de asignación

- Una variable en el lado derecho de una sentencia de asignación debe tener un valor antes de que la sentencia de asignación se ejecute. Hasta que un programa le da un valor a una variable, esa variable no tiene valor definido.

```
int x,y;
y = x + 1;
```

ERROR LÓGICO: *la variable **x** no tiene ningún valor. El valor que toma la variable **y** es impredecible!!!*

- En la izquierda de una sentencia de asignación solo pueden existir variables. La siguiente expresión no es válida:

```
int Valor_Neto, Tasas;
Valor_Neto - Tasas =  34015; //ERROR!!!
```

Regla de asignación

- La operación de asignación es una operación destructiva
- El valor almacenado en una variable se pierde/destruye/sustituye por el nuevo valor en la asignación

Constantes

- Son datos cuyo contenido no varía a lo largo de la ejecución del programa. Es un nombre que se da a un valor literal.

```
const double PI=3.1415;
const int SALARIO_BASE=1000;
const bool VERDAD=true;
```

- Suelen usarse identificadores (nombres) sólo con mayúsculas para diferenciarlos de las variables
- Ventajas de la declaración de constante
  - *Proporcionan información*
  - *Imposibilidad de cambiarlo por error (PI)*
  - *Posibilidad de cambios futuros (SALARIO_BASE)*

Valores, literales y constantes

```c
// Calculo del area de una circunferencia
#include <stdio.h>

int main() {
    const double PI = 3.141516;
    double radio, rta;
    radio = 8.73;
    rta = PI * radio * radio;

    printf("El valor del area es: %f\n", rta);
    return 0;
}
```

Declaración de Constantes

Declaración de Variables

Un literal en una asignación

Una expresión en una asignación

Identificadores

Para asignar un ***identificador*** deben respetarse las siguientes reglas:

- Son una serie de caracteres restringidos (letras, dígitos, y subrayado '_')
- No pueden comenzar por un número
- No pueden contener acentos, ni tampoco espacios en blanco
- No se puede utilizar la letra 'ñ' ni tampoco las barras / y \
- Sensibles a mayúsculas y minúsculas (Alto ≠ alto)
- No puede haber dos identificadores iguales en el mismo ámbito ({...})

Normas de nombrado de variables

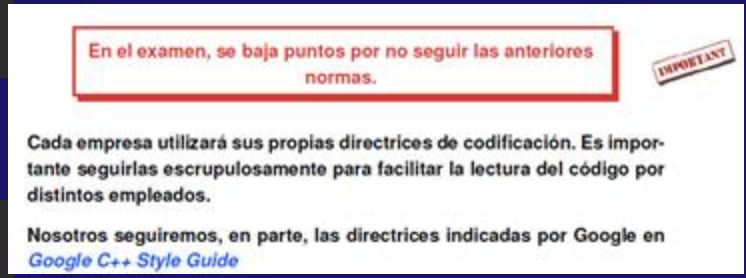
- El identificador de un dato debe reflejar su contenido (semántica)

```
float v, l, a; //MAL!
float voltaje, lado1, lado2; //BIEN ☺
```

- No usamos nombres genéricos

```
int aux, temp; //MAL!
int copia, calificacion; //BIEN ☺
```

- Usamos minúsculas para los datos variables. Las constantes en mayúsculas



https://google.github.io/styleguide/cppguide.html

Palabras reservadas (tokens)

Existen "palabras reservadas" que no se pueden utilizar como identificadores. (dependen de cada lenguaje)

Palabras reservadas comunes a C (C89) y C++

auto	break	case	char	const	continue	default
do	double	else	enum	extern	float	for
goto	if	int	long	register	return	short
signed	sizeof	static	struct	switch	typedef	union
unsigned	void	volatile	while			

Palabras reservadas adicionales de C++

and	and_eq	asm	bitand	bitor	bool	catch
class	compl	const_cast	delete	dynamic_cast	explicit	export
false	friend	inline	mutable	namespace	new	not
not_eq	operator	or	or_eq	private	protected	public
reinterpret_cast	static_cast	template	this	throw	true	try
typeid	typename	using	virtual	wchar_t	xor	xor_eq

Palabras reservadas añadidas en C99

_Bool	_Complex	_Imaginary	inline	restrict

Compatibilidad de tipos

Intentar asignar a las variables valores de su mismo tipo.

No obstante, los valores de tipo int pueden almacenarse en variables de tipo double.

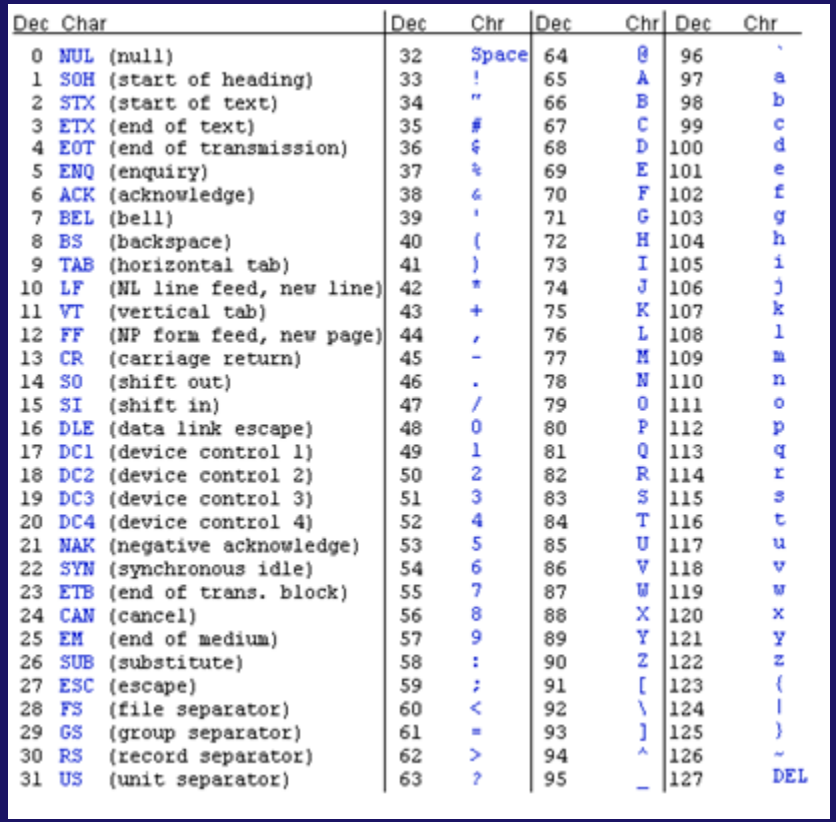
```
int unEntero;
double unReal;
unEntero = 18;
unReal = unEntero; //unReal vale 18.0
unReal = 98;       //unReal vale 98.0
```

La asignación de valores de tipo double a variables de tipo int provoca el truncamiento de la parte decimal.

```
int unEntero;
unEntero = 2.456; //unEntero vale 2
```

Más sobre el tipo char

- char es un conjunto finito y ordenado de caracteres:
  - Letras minúsculas, mayúsculas, dígitos del 0 al 9 y otros caracteres especiales.
- 1 char = 1 byte = 256 valores diferentes.
- Cada valor tiene un símbolo asociado en la tabla ASCII.



Dec	Char	Dec	Chr	Dec	Chr	Dec	Chr
0	NUL (null)	32	Space	64	@	96	`
1	SOH (start of heading)	33	!	65	A	97	a
2	STX (start of text)	34	"	66	B	98	b
3	ETX (end of text)	35	#	67	C	99	c
4	EOT (end of transmission)	36	$	68	D	100	d
5	ENQ (enquiry)	37	%	69	E	101	e
6	ACK (acknowledge)	38	&	70	F	102	f
7	BEL (bell)	39	'	71	G	103	g
8	BS (backspace)	40	(	72	H	104	h
9	TAB (horizontal tab)	41	)	73	I	105	i
10	LF (NL line feed, new line)	42	*	74	J	106	j
11	VT (vertical tab)	43	+	75	K	107	k
12	FF (NP form feed, new page)	44	,	76	L	108	l
13	CR (carriage return)	45	-	77	M	109	m
14	SO (shift out)	46	.	78	N	110	n
15	SI (shift in)	47	/	79	O	111	o
16	DLE (data link escape)	48	0	80	P	112	p
17	DC1 (device control 1)	49	1	81	Q	113	q
18	DC2 (device control 2)	50	2	82	R	114	r
19	DC3 (device control 3)	51	3	83	S	115	s
20	DC4 (device control 4)	52	4	84	T	116	t
21	NAK (negative acknowledge)	53	5	85	U	117	u
22	SYN (synchronous idle)	54	6	86	V	118	v
23	ETB (end of trans. block)	55	7	87	W	119	w
24	CAN (cancel)	56	8	88	X	120	x
25	EM (end of medium)	57	9	89	Y	121	y
26	SUB (substitute)	58	:	90	Z	122	z
27	ESC (escape)	59	;	91	[	123	{
28	FS (file separator)	60	<	92	\	124	|
29	GS (group separator)	61	=	93	]	125	}
30	RS (record separator)	62	>	94	^	126	~
31	US (unit separator)	63	?	95	_	127	DEL

char como entero pequeño

- En C, el tipo char es realmente un tipo entero pequeño.
- Cuando hacemos

```
char letra_piso = ‘A’;
```

estamos asignando a la variable letra_piso el número de orden en la tabla ASCII del carácter 'A', es decir, el 65.

```
int letra_piso;
letra_piso = 'A'; // Almacena 65
letra_piso = 65; // Almacena 65
letra_piso = '7'; // Almacena 55
letra_piso = 7; // Almacena 7
```

char como entero pequeño

- Podemos operar con los caracteres como si fuesen números:

```
char letra_piso; // También valdría int letra_piso;
letra_piso = 'A'+1; // Almacena 66 => 'B'
letra_piso = 65+1; // Almacena 66 => 'B'
letra_piso = '7'- 1; // Almacena 54 => '6'
letra_piso = 'A'+'7'; // Almacena 120 => 'x'
letra_piso = 325;  //??????????
```

- También podemos usar el operador - con dos argumentos de carácter y obtener como resultado un entero

```
int orden;
orden = 'c' - 'a' // Almacena 2
```

El tipo enum

- Idea: crear un tipo personalizado con un rango de valores restringido:
  - Colores
  - Opciones de un menú
  - Etc.
- Definiremos los posibles valores con palabras
- Estos nuevos valores (las palabras) solo se pueden usar con las variables del tipo enum definido
- Usaremos nombres con mayúsculas para el nombre de la enumeración

```
enum Color{rojo, verde, azul};
```

Ejemplo de enum

- Podemos asignar cualquiera de los valores a la variable enum creada
- También pueden usarse los valores definidos del enum en comparaciones

```
enum Color{rojo, verde, azul};


int main() {
    enum Color mi_color, otro_color;

    mi_color = rojo;
    otro_color = azul;

}
```

Los valores de enum

- ¿Qué imprime el siguiente código?

```
enum Color mi_color;
mi_color = rojo;
printf("%d\n", mi_color);
```

Los valores de enum

- De forma interna, cada valor definido del tipo enum es un **entero,** con un valor que se asigna de las siguiente manera:
  - El primer valor definido se corresponde al valor 0
  - Un valor cualquiera del enum se corresponde con un valor igual al entero del valor previo más 1 (+ 1).

```
enum Color mi_color;
mi_color = rojo;
printf("%d\n",mi_color);
```



Salida de consola
0

```
enum Color mi_color;
mi_color = azul;
printf("%d\n",mi_color);
```



Salida de consola
2

```
enum Color mi_color;
mi_color = azul;
printf("%d\n", azul);
```



Salida de consola
ERROR

03

Entrada y salida de datos

Operadores: entrada y salida

- La **entrada** se realiza con la función scanf:
  - scanf utiliza la entrada estándar (teclado del PC) para captar el valor y se le proporciona la variable en la que almacenar el valor precedida de &
  - Además, debemos indicar el tipo de dato que esperamos entre comillas dobles

```
scanf("%d", &variable); //dato numérico “d”
```

- La **salida** se realiza con la función printf:
  - printf necesita conocer el tipo de dato que imprimimos entre comillas dobles
  - Debemos proporcionar la variable SIN &

```
printf("%d", variable); //dato numérico “d”
```

Salida de datos

- Los valores de las variables, así como cualquier texto, se puede mostrar por pantalla a usando printf
- Su sintaxis más elemental es:

```
printf("mensaje"); //imprime “mensaje”
```

- Es posible combinar la impresión de cadenas de texto con variables o cálculos usando el carácter % para el formato y proporcionando los datos a imprimir

```
printf("un calculo %d, otro calculo: %d", 2+3, mi_resultado);
```

- Existen *secuencias de escape* que permiten dar un formato más “estilizado” a la salida.

Salida de datos: formato

Secuencia	Significado
\n	nueva línea
\t	tabulador
\b	backspace
\r	retorno de carro
\"	comillas
\'	apostrofe
\\	backslash
\?	signo de interrogación

Más sobre printf

```c
#include <stdio.h>

int main() {
    printf("Dame dos números ");


    printf("Dame");
    printf(" dos ");
    printf("números ");

    return 0;
}
```

Son equivalentes

Entrada de datos

- La función scanf permite ingresar datos desde el teclado.
- El valor leído debe almacenarse en una variable.
- Su sintaxis es similar a cout pero hay que incluir el ampersand (&) en la variable:

```
scanf("%d", &mi_variable);
```

- scanf Espera a que el usuario introduzca un valor entero desde el teclado y, cuando se pulsa la tecla Intro, lo almacena en mi_variable

- Es muy importante usar scanf junto con printf:

```
printf("Introduce un número: ");
scanf("%d", &mi_variable);
```

Ejemplo

```c
// Programa que lee dos números enteros por teclado
// y muestra por pantalla su suma
#include <stdio.h>

int main() {
    int variable1, variable2;
    int suma;

    printf("Introducir primer dato: ");
    scanf("%d", &variable1);
    printf("Introducir segundo dato: ");
    scanf("%d", &variable2);

    suma = variable1 + variable2;

    printf("La suma es: %d\n", suma);
    return 0;
}
```

```
Introducir primer dato: 45
Introducir segundo dato: 72
La suma es: 117
```

Leyendo enumeraciones

- ¿Podemos leer enumerados? ¿Qué puede ocurrir en el siguiente fragmento de código?

```
enum Color mi_color;
scanf("%d", &mi_color);
```

Ejercicios

1. ¿Cuales de los siguientes son nombres válidos de variables?
x_1 x1 12342 _hh %valor prog.cpp
2. ¿ Que hace el siguiente ejemplo?

```
int valor;
valor = 1;
valor = valor + 1;
printf("%d\n", valor);
```

3. ¿Salida del programa siguiente?

```
char a,b,c;
a='b';
b='c';
c = a;
Imprimir → a,b,c,'c';
```

04

Expresiones y operaciones

Operadores aritméticos

- Se pueden utilizar con valores númericos (int, double...).
- Operandos numéricos de distinto tipo: el tipo del resultado será el tipo del operando con un rango más amplio:
  - `int` OP `int` → `int`
  - `int` OP `double` → `double`

- Operadores
  - `+` Suma
  - `-` Resta
  - `*` Multiplicación
  - `/` Division
  - `%` Modulo *(¡sólo para enteros!)*



División

Entre enteros

Trunca la parte decimal del resultado

- 10/3 → 3
- 5/2 → 2

Módulo

Resto de la división entera

- 7%5 → 2

Un operando es real

El resultado es de tipo real y tiene decimales

- 10.0/3 → 3.33333
- 5/2.0 → 2.5

Reglas de precedencia

- Determinan el orden de evaluación de los operadores
- A igual preferencia, se evalúa de izquierda a derecha

a+b/c ≠ (a+b)/c

	Operator Precedence
1	! Logical not (Highest)
2	( ) Parenthesis
3	*, /, %
4	+, -
5	>, >=, <, <=
6	==, !=
7	&& (AND)
8	|| (OR)
9	= (Lowest)

Funciones matemáticas predefinidas

- Llevan a cabo cálculos matemáticos y devuelven un valor
- La forma para invocarlos es:

```
nombre_funcion(argumento1, argumento2, ...)
```

```
double raizCuadrada = sqrt(900);
double coseno = cos(45);
int redondeo = floor(3.4);
```

- Los argumentos pueden ser:
  - *Literales:* `sqrt( 4 )`
  - *Variables:* `sqrt( x )`
  - *Expresiones:* `sqrt( sqrt( x ) )` ó `sqrt( 3 - 6x )`
- Importante:
  - Saber el número de entradas y su tipo.
  - Saber el tipo del valor que devuelve

Bibliotecas de funciones

- Las funciones se almacenan en **bibliotecas**.
- Ejemplos:
  - Biblioteca <math.h>: sqrt, pow, ceil, floor, cos, sin, tan, exp, log, log10, abs, fabs
  - Biblioteca <ctype.h>: isalnum, isalpha, isdigit, islower, isupper, tolower, toupper

- Toda función se identifica con su prototipo (tipo de salida, identificador, tipo de cada entrada):

```
double sqrt(double)
```

Funciones matemáticas populares

Numero	Descripcion	Ejemplo
ceil(x)	redondea x al entero mas pequeño no menor que x	ceil(9.2) es 10.0 ceil(-9.8) es –9.0
floor(x)	redondea x al entero mas grande no mayor que x	floor(9.2) es 9.0 floor(-9.8) es –10.0
fabs(x)	valor absoluto de x	fabs(-9.5) es 9.5
fmod(x,y)	residuo de x/y como numero de punto flotante	fmod(13.657, 2.333) es 1.992
sqrt(x)	raiz cuadrada de x	sqrt(9.0) es 3.0
pow(x,y)	x elevado a la potencia y	pow(2,7) es 128
log(x)	logaritmo natural de x(base e)	log10(x)
log10(x)	logaritmo de x(base 10)	log(2.718282) es 1
exp(x)	funcion exponencial	exp(0.1) es 2.71828
sin(x)	seno trigonometrico de x(en radianes)	sin(0.0) es 0.0
cos(x)	coseno trigonometrico de x(en radianes)	cos(0.0) es 1
tan(x)	tangente trigonometrico de x(en radianes)	tan(0.0) es 0

Ejercicio

Hacer un programa que pida al usuario la longitud de dos catetos de un triángulo rectángulo y que calcule la hipotenusa.

Pasos a seguir:

- Diseño: ¿Cómo se calcula la hipotenusa?
- ¿Cuántos datos constantes y variables necesitamos, y de qué tipo?
- ¿Qué bibliotecas necesitamos y cómo se incluyen?
- Pasos: Entrada de datos, procesamiento, salida de datos
- ¡No olvidar poner comentarios!

Ejercicio

- Supongamos que un producto cuesta 17345 euros. ¿A cuántas pesetas equivale?. Implemente un programa que permita contestar a esta pregunta. (1 euro = 166.386 pesetas)
- ¿Cómo implementar un programa que permita hacer la transformación de cualquier cantidad de euros a pesetas?
- Implemente un programa que lea un valor entero, y muestre el doble y el triple de su valor

Ejercicio

- Convierta las siguientes fórmulas a expresiones en C: 3x

  3x+y $\frac{x+y}{7}$ $\frac{x+y}{z+2}$

- Escriba un programa que dados dos puntos p1 = (x1,y1) y p2=(x2,y2), calcule la distancia Euclídea entre ambos

Operadores de actualización

- En C, también están disponibles los operadores **+=, -=, *=, /=, %=, ++, --**, que modifican el valor de una variable de forma relativa a su valor actual:

Oper.incremento

¿Diferencia entre x++ y ++x?

Operador	Ejemplo	Expresión extendida
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
***=**	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 3	X = x % 3
++	x++	x = x + 1
++	++x	x = x + 1
--	x--	x = x - 1
--	--x	x = x - 1

Operador incremento

```c
#include <stdio.h>

int main() {
    int y = 4;

    y++;
    printf("Ahora y vale %d\n", y);
    printf("Ahora y vale %d\n", ++y);
    printf("Ahora y vale %d\n", y++);
    printf("Ahora y vale %d\n", y);

    return 0;
}
```

Operadores lógicos

- Son operadores binarios
- Se aplican sobre datos (y expresiones) de tipo bool:
  - AND (&&), OR (||)
  - NOT (!) (este es un operador unario)
- Devuelven true o false

AND	True	False
True	True	False
False	False	False

OR	True	False
True	True	True
False	True	False

NOT

True	False
False	True

Leyes de De Morgan

```
!(X || Y) → !X && !Y
!(X && Y) → !X || !Y
```

Operadores relacionales

- Son los operadores habituales de comparación de números.
- El resultado es de tipo bool.

```
(4 < 5)   ⇒ true
(4 > 5) ⇒   false
(1 < x < 10) ⇒   ERROR LÓGICO
((x >= 1) && (x <= 10)) ⇒ ¿x ∈ [1, 10]?
```

Operación	Operador	Ejemplo	Comparación	Resultado/Binario
Mayor que	>	X > Y	¿Es X mayor que Y?	Verdadero (1)
Menor que	<	X < Y	¿Es X menor que Y?	Falso (0)
Mayor o igual que	>=	X >= Y	¿Es X mayor o igual que Y?	Verdadero (1)
Menor o igual que	<=	X <= Y	¿Es X menor o igual que Y?	Falso (0)
Igual a	==	X == Y	¿Es X igual a Y?	Falso (0)
Diferente a	!=	X != Y	¿Es X diferente a Y?	Verdadero (1)

Ejercicio

- Dadas las variables count = 0 y limit = 10, calcule el valor de las siguientes expresiones booleanas:

```
(count == 0) && (limit < 20)

(limit > 20) || (count < 5)

!(count == 12)

(count == 1) && (x < y)

!(((count < 10) || ( x < y )) &&
    (count >= 0))
```

Ejemplo

- A es menor o igual que B y B no es mayor que C

```
int A = 40, B = 34, C = 50;
bool condicion;
condicion = A <= B && !B > C; // MAL
condicion = (A <= B) && (!(B > C)); // BIEN
condicion = (A <= B) && !(B > C); // Recomendado
condicion = (A <= B) && (B <= C); // Mucho mejor
```

Simplificar las expresiones lógicas, para así aumentar su legibilidad.

Ejercicios expresiones lógicas

- Obtener true si un número es positivo
- Obtener true si una pareja de números son ambos negativos
- Obtener true si un número es par
- Obtener true si un carácter está en mayúsculas
- Obtener true si un número de teléfono (representado como un entero) pertenece la provincia de Granada
- Tomar tres números y obtener true si el 3º es la suma de los dos 1ºs
- Obtener true si un año es bisiesto
- Dados tres valores (día, mes, año) obtener true si la fecha es correcta
- Dadas las coordenadas de un rectángulo (esquina superior izquierda e inferior derecha) y un punto, obtener true si está dentro del anterior rectángulo.

05

Consejos prácticos y errores comunes

Consejos prácticos

- Escribir programa simples y claros.
- Comenzar los programas con un comentario que describa su propósito.
- Colocar un espacio después de cada coma (,).
- Asignar nombres significativos a las variables.
- Colocar una línea en blanco entre bloques de código: (entrada frente a cómputo, cómputo frente a salidas, etc.)
- Colocar espacios en cada lado de un operador binario.
- No incluir más de una instrucción por línea.
- Terminar cada programa con \n o endl.

Errores comunes

- No inicializar las variables
- Usar datos enteros y aplicar una división real
- Cadenas de desigualdades (x<z<y)
- Olvidar incluir stdio.h en un programa que usa scanf o printf
- Omitir el punto y coma al final de una instrucción
- Usar el módulo (%) sobre operandos no enteros
- Dejar espacios en blanco en mitad de los operadores ==, !=, <= y >=
- Confundir el operador de igualdad == con el de asignación =
- Partir los identificadores con espacios en blanco (main)